An MLIR-Based Approach to HPC Portability with Mojo

Yannick Schürmann

***Abstract*—High-performance computing faces two persistent challenges. The first is the two-language problem, where researchers prototype in Python but must rewrite performance-critical code in C++ or CUDA to meet performance demands. The second is vendor lock-in, where GPU code written for NVIDIA hardware cannot run on AMD or other architectures without significant porting effort. Prior approaches have addressed each problem in isolation, through portability layers such as Kokkos and SYCL, or through high-level languages such as Julia, but none have resolved both simultaneously. Mojo is a new programming language built directly on LLVM's Multi-Level Intermediate Representation (MLIR), which allows it to address both problems at the compiler level rather than through libraries or workarounds. This paper examines Mojo's design, analyzing how its MLIR foundation enables Python interoperability and vendor-portable GPU programming, and evaluates its HPC viability through benchmarks conducted by researchers at Oak Ridge National Laboratory on NVIDIA H100 and AMD MI300A GPUs. Results show that Mojo performs competitively with vendor baselines for memory-bound workloads, while gaps remain for compute-bound kernels due to missing fast-math compilation support. Although practical limitations remain, Mojo represents a novel approach to both challenges at the compiler level that may prove more capable than existing alternatives as the ecosystem matures.**

I. Introduction

The rise of AI has only accelerated demand for raw compute, with training and inference workloads now rivaling traditional HPC simulations in scale. Yet the software stack forces developers to choose between writing portable high-level code that sacrifices performance and writing low-level kernels locked to a single vendor's hardware. Mojo, discussed in this paper, aims to solve both the afformentioned problems.

The choice is forced by two issues, vendor lock-in and the two-language problem. The first issue, vendor lock-in, is that CUDA, NVIDIA's proprietary programming language and the standard for GPU programming [1], does not target other vendors. As compute demand grows, hardware diversifies. For example, Frontier, the first exascale machine, runs entirely on AMD GPUs [2]. AMD provides its own counterpart to CUDA's ecosystem, the Radeon Open Compute (ROCm) software stack [3], within which the Heterogeneous-Compute Interface for Portability (HIP) is the CUDA-equivalent language used to write GPU kernels [4]. On non-NVIDIA hardware, CUDA kernels cannot run without substantial porting, and would have to be rewritten in HIP. The second issue, the two-language problem, is that when writing HPC code, researchers often prototype in high-level languages such as Python, but to reach performance demands they must rewrite hot paths in systems-level languages like C++ [5,6], which greatly impacts productivity.

These two problems demand expertise across multiple ecosystems, increase maintenance burden, and slow the path from research to production. Julia eliminates the two-language problem in a high-level language through just-in-time (JIT) compilation to native performance, yet remains vendor-dependent [7]. Kokkos and SYCL offer hardware portability but stay within the C++ ecosystem, so they still suffer from the language split issue [8,9]. No existing approach addresses both simultaneously. Solving both problems would allow HPC developers to write productive, high-performance code in a single language, without sacrificing portability across hardware vendors.

Multi-Level Intermediate Representation (MLIR) is a compiler infrastructure designed to operate across multiple abstraction levels simultaneously, rather than targeting one fixed level like LLVM Intermediate Representation (LLVM IR) [10]. Our running example, a matrix multiplication, can be expressed as a high-level linear algebra operation and lowered through loop and memory-layout optimizations down to vendor-specific GPU instructions.

Built on this compiler infrastructure, Mojo is a novel attempt at solving both issues [11]. It is designed to become a superset of Python, adding features such as ownership, static typing, and compile-time metaprogramming, which give the compiler the information it needs. Without static types, the compiler cannot lower through abstraction levels. Without ownership, GPU memory cannot be managed safely without a garbage collector, which HPC workloads cannot afford. Without compile-time metaprogramming, problem sizes and layouts remain unknown until runtime, preventing the specialization MLIR depends on. Together these properties allow Mojo to compile high-level, vendor-agnostic code to optimized, vendor-specific GPU instructions. This paper examines Mojo's design and evaluates its HPC viability through analysis of benchmarks conducted by researchers at Oak Ridge National Laboratory against CUDA and HIP on NVIDIA and AMD GPUs.

We begin with the background on LLVM IR and MLIR,

establishing the compiler foundation Mojo builds on. We then take a look at prior approaches and their limitations to the two problems. From there we cover how Mojo is implemented as a set of MLIR dialects and what that means for language features and hardware portability. Finally we go through real HPC benchmarks on NVIDIA and AMD GPUs, discuss what the results tell us about Mojo's current strengths and gaps, and consider its future trajectory.

II. Background

Mojo is implemented directly on top of MLIR, which itself builds on LLVM IR. Understanding how IR works, where it falls short, and how MLIR addresses those shortcomings gives the context for Mojo's key properties, which let it address both the two-language split and vendor lock-in.

A. LLVM

LLVM is a compiler framework designed to normalize code representation across languages, making it possible to apply a shared set of optimizations to any code compiled with it [12]. To achieve this, LLVM represents programs in Static Single Assignment (SSA) form, where each variable is defined exactly once. This representation, called LLVM Intermediate Representation (LLVM IR), abstracts away language-specific constructs and reduces code to raw semantics, allowing the same optimizations to work across C, C++, Rust, Swift, and many other languages. IR is often described as roughly C with vectors [10], a single fixed IR. This means both high-level language features and machine-specific details are out of scope. To illustrate, consider the matrix multiplication below.

```
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

Figure 1: Matrix multiplication in C.

Compiled to IR, the loop structure disappears entirely into conditional branch instructions and jump labels. What remains is raw pointer arithmetic operating on individual memory addresses. Each % assignment is an instance of SSA. The snippet below shows only the inner loop body as the full IR is significantly longer, with the loop bounds and iteration logic expressed as branches elsewhere.

The compiler can optimise individual operations but has lost the information that this is a matrix multiply, what the memory layout is, or that the loops are parallelisable. This matters because higher-level optimisations such as loop tiling for cache efficiency, parallelisation across GPU threads, or mapping to specialised hardware instructions all depend on that structural knowledge. Within LLVM IR, those optimisations are no longer possible.

As stated in the original LLVM paper, LLVM is not intended to be a universal compiler IR and does not

```
%a_p  = getelementptr float, float* %A, i64 %a_i
%a_v  = load float, float* %a_p
%b_p  = getelementptr float, float* %B, i64 %b_i
%b_v  = load float, float* %b_p
%prod = fmul float %a_v, %b_v
%c_p  = getelementptr float, float* %C, i64 %c_i
%c_v  = load float, float* %c_p
%sum  = fadd float %c_v, %prod
store float %sum, float* %c_p
```

Figure 2: Matrix multiplication from Figure 1 in LLVM IR.

represent high-level language features directly, nor does it capture hardware-specific instruction patterns such as SIMD intrinsics or target-specific addressing modes [12]. This works well for CPU-targeted, homogeneous compilation but becomes a limiting factor when targeting heterogeneous hardware or HPC domain-specific optimizations. Many problems are simply better modeled at a different abstraction level. Source-level C++ analysis is one example, since constructs like classes and templates are erased before reaching LLVM IR, making them invisible to any IR-level tool. As a result, the compilers for Swift, Rust, and Julia each introduced their own mid-level IRs before targeting LLVM, and frameworks like TensorFlow contained over a dozen distinct compiler components each with their own IR, leading to fragmented and duplicated infrastructure with each team reimplementing the same class of optimizations [10], as illustrated in Figure 3 and Figure 4.

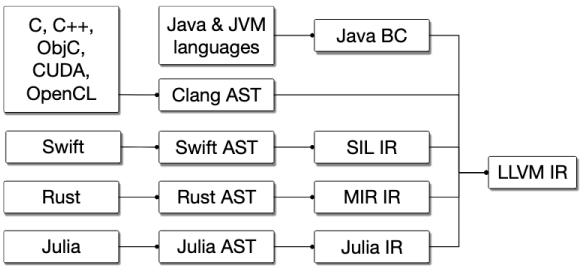

Figure 3: Compilation pipeline of different languages with multiple mid-level IRs for language-specific optimization with common backend for multiple hardware targets [10]

B. MLIR

1) Motivation

The fixed abstraction level of LLVM IR, while powerful for general-purpose compilation, creates a recurring problem. Whenever engineers needed to solve a domain-specific compiler challenge, they had no good place to put it. Problems such as memory layout decisions, for example whether to store a matrix in row-major or column-major order, are too specific for the source language but require structural knowledge that LLVM IR has already lost. This

led engineers to develop their own intermediate compilers, resulting in fragmented and duplicated infrastructure, particularly in ML and HPC compiler stacks. MLIR was developed to solve this by making it cheap to define new intermediate representations within a single, shared compiler infrastructure [10].

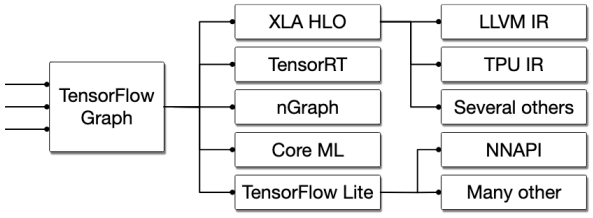

Figure 4: Fragmented TensorFlow stack [10]

2) Core Idea

At the core of MLIR is the principle of being multi-level, where multiple IRs can coexist within a single framework. A key property of this is that different parts of the same program can reside in different IRs simultaneously, allowing high-level structure to be preserved where it is needed for optimization while other parts are already lowered to hardware-specific instructions. This is captured by MLIR's guiding design philosophy of "little builtin, everything customizable". The system is built on a small set of fundamental concepts, namely types, operations, and attributes, with most of the intermediate representation left open for extension. A key measure of success for this approach is the ability to express a diverse set of abstractions, from machine learning graphs and abstract syntax trees all the way down to instruction-level representations like LLVM IR, all without hardcoding any of these concepts into the system. This stands in direct contrast to LLVM, which provided only a single fixed abstraction level [10]. This lets compiler engineers work at the right level of detail for a problem without building a separate compiler infrastructure.

3) Dialects

Dialects are the primary mechanism through which MLIR achieves its extensibility. A dialect is a logical grouping of operations, types, and attributes under a unique namespace, and adding a new dialect is how compiler engineers introduce new abstractions into the system [10]. The namespace appears as a dot-separated prefix on each operation, as can be seen in Figure 5 below.

```
linalg.matmul
  ins(%A, %B : memref<?x?xf32>,
               memref<?x?xf32>)
  outs(%C : memref<?x?xf32>)
```

Figure 5: Matrix multiplication expressed in MLIR's *linalg* dialect.

The separation into dialects is modular by design, and operations from different dialects can coexist at any level of the IR at any time, allowing them to be mixed and progressively lowered, each step preserving only what is still needed [10].

This composability also enables reuse across domains. An OpenMP dialect, for example, can be shared between Fortran and C language frontends rather than each implementing their own parallel constructs [10]. Similarly, hardware vendors can contribute target-specific dialects such as NVVM for NVIDIA GPUs or AMDGPU for AMD, making hardware portability a property of the compiler infrastructure rather than something each language or framework has to solve on its own.

4) Progressive Lowering

Progressive lowering is the principle that a high-level representation should be compiled to hardware-specific instructions not in a single step, but gradually through a series of small transformations across multiple IRs, each step referred to as a lowering [10]. The need for this stems from the variety of platforms and programming models a generic compiler infrastructure has to support, as the same source program may need to target a CPU, a GPU, or a custom accelerator, each requiring different lowering paths.

To see what this looks like in practice, consider the matrix multiplication example again. In MLIR, this can be expressed at the high-level *linalg* dialect and then progressively lowered through *affine* (loop structure), where loop bounds and memory indices are constrained to affine functions of enclosing loop variables, then *scf* (structured control flow), *cf* (control flow), and finally to LLVM IR for code generation. The *affine* form is shown in Figure 6.

```
affine.for %i = 0 to %M {
  affine.for %j = 0 to %N {
    affine.for %k = 0 to %K {
      %a    = affine.load %A[%i, %k]
                : memref<?x?xf32>
      %b    = affine.load %B[%k, %j]
                : memref<?x?xf32>
      %c    = affine.load %C[%i, %j]
                : memref<?x?xf32>
      %prod = arith.mulf %a, %b : f32
      %sum  = arith.addf %c, %prod : f32
      affine.store %sum, %C[%i, %j]
                : memref<?x?xf32>
    } } }
```

Figure 6: Matrix multiplication after lowering from the single *linalg* operation in Figure 5 to the *affine* dialect.

This stands in contrast to rigid compiler pipelines like Clang, which lowers from ASTs to LLVM IR, to SelectionDAG, to MachineInstr, and to MCInst in a fixed sequence where each step is destructive. Information from the previous representation is lost and cannot be recovered at a later stage. The key property that sets MLIR apart is

that different parts of the IR can sit at different abstraction levels simultaneously. As the paper describes, "mixing different levels of abstractions and different concepts in the same IR is a key property of the system to allow a part of the representation to remain in higher-level abstraction while another part is lowered" [10]. This means a compiler for a custom accelerator can reuse high-level loop structure alongside primitive scalar/vector instructions specific to that hardware, all within the same IR at the same time.

5) SSA and Nested Regions

MLIR extends SSA with *nested regions* as a first-class concept. In MLIR, a region is a structural IR concept, an ordered list of basic blocks scoped inside an operation, used to organize control flow and value visibility rather than to manage memory. Operations can contain regions, which in turn contain blocks and further operations, forming a recursive structure. This allows higher-level abstractions like loops, closures, and parallel constructs to live directly in the IR at any abstraction level [10]. This structure is illustrated in Figure 7.

```
%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.        Region
 ^block(%argument: !d.type):                                   Block
   // Ops have function types (expressing mapping).
   %value = "nested.operation"() ({
     // Ops can contain nested regions.                         Region
     "d.op"() : () -> ()
   }) : () -> (!d.other_type)
   "consume.value"(%value) : (!d.other_type) -> ()
 ^other_block:                                                 Block
   "d.terminator"() [^block(%argument : !d.type)] : () -> ()
 })
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)
```

Figure 7: Operation (Op) is the main entity in MLIR. Operations contain a list of regions, regions contain a list of blocks, blocks contain a list of Ops, enabling recursive structures [10].

In the *affine* dialect of our running example (Figure 6), each loop body lives inside the operation as a region rather than as a separate function. The compiler can therefore analyse and transform the loop structure, for tiling, parallelisation, or vectorisation, without losing the surrounding context [10].

6) Why This Matters for Portability

The properties described above come together to directly enable hardware portability. Because MLIR's IRs are defined by dialects, the same high-level representation can be compiled to different hardware backends by selecting a different set of lowering passes at compile-time, each targeting the appropriate hardware dialect. A matrix multiplication expressed at the *linalg* level can be lowered to NVVM for NVIDIA GPUs or AMDGPU for AMD without any changes to the source representation, keeping the code hardware-agnostic at the top and hardware-specific only at the final lowering step. This is the architectural foundation that allows Mojo, as the first language built directly on MLIR, to inherit vendor portability as a compiler property rather than a library concern. MLIR also supports both ahead-of-time compilation, where code is fully compiled before execution, and just-in-time compilation, where code is compiled at runtime. Which optimizations are available depends on when information is known to the compiler. Optimizations such as loop unrolling, memory layout specialization, and register allocation require types and problem sizes to be fixed at compile-time, and are only possible under ahead-of-time compilation or when just-in-time compilation can observe them early enough. This distinction becomes relevant when evaluating Mojo's design and comparing it with approaches like Julia, which are covered in the following sections.

III. Prior Work

Prior work has addressed each problem in isolation. The C++ ecosystem produced a series of portability layers, including OpenCL, SYCL, and Kokkos, that abstract over vendor hardware at the cost of remaining within C++. Julia took a different approach, eliminating the language split through JIT compilation to near-native performance, but without resolving vendor lock-in. No existing solution addresses both problems simultaneously, as summarized in Table I.

Table I: Summary of approaches and the problems they address.

Approach	Vendor Portability	Two-Language Problem
OpenCL	Yes	No
SYCL	Yes	No
Kokkos	Yes	No
Julia	Partial	Yes

A. OpenCL

OpenCL is an open framework for heterogeneous computing that provides a common API for executing parallel code across CPUs, GPUs, and accelerators from different vendors [13]. By defining a shared device model and kernel interface, it allows the same code to run correctly on any conforming hardware, directly addressing vendor lock-in. However, correctness and performance are separate concerns, and achieving good results on a given target architecture still requires platform-specific tuning of memory layouts and work-group sizes. Because kernels are written in a C-based language managed through a verbose host API [13], the gap between high-level code and low-level GPU kernels remains fully intact.

B. SYCL

SYCL is a C++ programming model built on top of OpenCL that retains its cross-vendor portability while significantly improving developer ergonomics [9]. Through single-source programming, host and device code coexist

in the same file using standard C++ constructs, removing the need for separate kernel strings and the verbose host API that OpenCL exposes. Because the language is still C++, the two-language problem remains unaddressed.

C. Kokkos

Kokkos is a C++ library for performance-portable parallel programming, designed to run efficiently across NVIDIA and AMD GPUs, multicore CPUs, and custom accelerators. Where OpenCL and SYCL address functional portability, Kokkos makes performance portability a first-class concern through its type-level abstractions. Kernels written once compile transparently to CUDA, HIP, or OpenMP depending on the target [8]. Because Kokkos is a C++ library, it shares the same limitation as the approaches above, leaving the two-language problem unaddressed.

D. Julia

Julia is a high-level, dynamically typed language for technical computing [5] that addresses the two-language problem through JIT compilation, specializing code on runtime types to achieve near-native performance competitive with C and Fortran [7]. Its compiler internally lowers code through multiple IR levels, from a high-level Julia IR down to LLVM IR, an approach that resembles multi-level compilation but without standardized extensibility across hardware targets. GPU kernels can be expressed in the same language through packages such as CUDAnative.jl, removing the need to switch to a lower-level language entirely. This GPU support was built around NVIDIA's CUDA toolkit however, and support for other vendors exists as separate, independently maintained packages, leaving vendor portability considerably less resolved than in the C++ approaches above.

IV. Mojo

Mojo is the first language fully built on MLIR, implemented as a set of MLIR dialects rather than targeting MLIR from outside [11]. Where the approaches in the previous section each addressed one problem in isolation, Mojo's MLIR foundation enables both solutions in a single language. Python interoperability closes the language split, and MLIR's hardware dialects fix the portability issue without external libraries or separate compilation pipelines.

A. Resolving the Two-Language Problem

Mojo is designed as a superset of Python, with the goal that any valid Python program is also a valid Mojo program. Existing Python libraries such as numpy and scipy can be imported and used directly without FFI boilerplate [14], so a developer can start from a working Python codebase and add performance incrementally.

Performance is controlled through type annotations on def functions. A def with no annotations behaves dynamically like Python, while the same def with type annotations compiles to typed MLIR ops and then to native code, making MLIR's progressive lowering user-facing without requiring a rewrite into a separate language [14]. Figure 8 shows this with a simple addition function, and the GPU kernel in Figure 9 applies the same principle at a larger scale.

```
def add(x, y):
    return x + y

def add(x: Float32, y: Float32) -> Float32:
    return x + y
```

Figure 8: The same def function without and with type annotations. The untyped version behaves dynamically like Python. The typed version compiles to typed MLIR ops and then to native code.

Python interoperability runs through CPython at runtime and is not compiled through MLIR. There is an explicit separation between the MLIR-compiled performance-critical code and Python interoperability, which remains a runtime-only concern [11]. As a result, the two-language problem is reduced but not fully resolved.

B. Resolving Vendor Lock-In

In Mojo, adding a new hardware target reduces to adding a lowering dialect in MLIR. NVIDIA GPUs are supported through the NVVM dialect and AMD GPUs through the AMDGPU dialect, meaning the same source compiles to either backend without modification. Where approaches like Kokkos and SYCL achieve portability by adding a library or abstraction layer on top of C++, Mojo's portability is a property of the compiler rather than a dependency [11].

C. Shared Foundations

Both resolutions are enabled by two shared language features, static memory layout and compile-time specialization.

Struct defines value types with explicit memory layout, mapping directly to MLIR's type system. Unlike Python classes, which allow dynamic dispatch and runtime field binding, Mojo structs are fully static and resolved at compile time [14], allowing MLIR to reason about GPU memory access patterns and generate optimised loads without additional annotations from the programmer.

Mojo also separates compile-time values, called parameters, from runtime values, called arguments. The compiler resolves parameters at compile time and generates a specialized version of the function for each unique set of parameter values [14], giving MLIR full visibility into problem shape and enabling loop unrolling, register specialization, and memory access optimisation for a specific kernel configuration.

Most HPC applications follow a compile-once, run-many-times approach where performance-critical information such as problem sizes and memory layouts is only known at

runtime. Mojo's reliance on MLIR optimizations requires this information to be fixed at compile-time, which can be a challenge for workloads where problem shape is determined dynamically, such as real-time simulations, adaptive mesh codes, or statistical methods [11].

D. GPU Programming Model

The two foundations above come together in Mojo's GPU programming model. Kernels are written with CUDA-like primitives (`block_idx`, `thread_idx`, `block_dim`) and dispatched through `DeviceContext`. `TileTensor` carries memory layout statically in its type, giving MLIR the information it needs to optimise memory access patterns at compile-time [11].

Figure 9 shows the same matrix multiplication from Figure 1 expressed in Mojo's GPU programming model. `comptime` values for `N`, `tile`, and `layout` are resolved by MLIR at compile-time, generating a specialised kernel for this exact problem shape.

```
...

comptime dtype     = DType.float32
comptime N         = 1024
comptime tile      = 16
comptime layout    = row_major[N * N]()
comptime num_tiles = ceildiv(N, tile)

def matmul_kernel(
    C: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    A: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
    B: TileTensor[dtype, type_of(layout),
                  MutAnyOrigin],
):
    var row = block_idx.y * block_dim.y
            + thread_idx.y
    var col = block_idx.x * block_dim.x
            + thread_idx.x
    if row < N and col < N:
        var acc = Scalar[dtype](0)
        for k in range(N):
            acc += A[row * N + k]
                 * B[k * N + col]
        C[row * N + col] = acc
```

Figure 9: Matrix multiplication from Figure 1 in Mojo's GPU programming model. `comptime` values fix problem size and layout at compile-time.

E. Benchmarking

We do not run our own benchmarks. Instead we survey the results reported by Godoy et al., who port four widely-used HPC proxy kernels to Mojo and compare them against CUDA on an NVIDIA H100 and HIP on an AMD MI300A [11]. The four kernels span two categories: two memory-bound (seven-point stencil and BabelStream) and two compute-bound (miniBUDE and Hartree-Fock). A kernel is *memory-bound* when its bottleneck is the rate at which data can be moved between memory and the processor, and *compute-bound* when the bottleneck is the arithmetic itself. Results are summarized using a performance portability metric Φ, defined as "the arithmetic mean of an application's performance efficiency observed across a set of platforms from the same architecture class" [11]

$$\Phi = \frac{\sum_{i \in T} e_i}{|T|}, \quad e_i = \frac{\text{Mojo}_{\text{perf},i}}{\text{vendor}_{\text{perf},i}}$$

where e_i is the ratio of Mojo's performance to the vendor baseline on platform i, and T is the set of GPU platforms tested, in this case the NVIDIA H100 and AMD MI300A. A value of $\Phi = 1.0$ indicates that Mojo matches vendor performance across all platforms.

Each kernel is measured using a metric suited to its nature. Memory-bound kernels are evaluated using effective memory bandwidth in GB/s, and compute-bound kernels using floating-point operations per second. Hartree-Fock is an exception, as its heavy use of atomic operations creates contention that makes throughput metrics misleading, so raw kernel execution time in milliseconds is used instead. Φ is still computed for all four kernels.

1) Memory-Bound Kernels

The seven-point stencil applies the Laplacian operator over a three-dimensional grid, a pattern common in PDE solvers for modeling diffusion phenomena. It is memory-bound because each output value requires reading seven neighbouring cells from global memory. Mojo reaches 82–87% of CUDA performance on the H100, with profiling showing that Mojo kernels use more registers per thread (24) than the equivalent CUDA kernel (21), which reduces cache efficiency and accounts for the gap [11]. On the AMD MI300A, Mojo performs on par with HIP, with $\Phi = 0.92$ across both platforms (Figure 11).

BabelStream measures the memory bandwidth limits of five standard array operations (Copy, Multiply, Add, Triad, and Dot). For the first four operations, Mojo slightly outperforms CUDA on the H100, a result attributed to MLIR generating fewer load constant operations than the equivalent CUDA kernel [11]. The Dot reduction, which requires shared memory operations, reaches only 78% of CUDA performance because the portable Mojo implementation does not replicate NVIDIA-specific heuristics used in the CUDA version. Results on the MI300A are nearly indistinguishable from HIP across all operations, with Φ = 0.96 (Figure 12).

2) Compute-Bound Kernels

miniBUDE simulates molecular docking, the computational process of predicting how a small molecule binds to a protein target. It is compute-bound due to the dense floating-point arithmetic involved. Mojo's performance is significantly affected by the absence of fast-math compilation support, which allows compilers to relax floating-point precision rules in exchange for speed. On the H100, Mojo sits between the non-fast-math and fast-math CUDA versions. On the MI300A, Mojo reaches only around 38% of HIP performance, giving $\Phi = 0.54$ [11] (Figure 13).

miniBUDE illustrates how Φ comes together, since it is the case where Mojo stays competitive with CUDA but falls well behind HIP. Each platform is measured at two configurations, and the per-run efficiencies are 0.82 and 0.59 against CUDA on the H100 and 0.38 and 0.38 against HIP on the MI300A (Figure 10). Averaging these four values gives

$$\Phi = \frac{0.82 + 0.59 + 0.38 + 0.38}{4} = 0.54$$

which shows how a single Φ collapses strong NVIDIA results and weak AMD results into one number that reflects neither platform on its own.

Hartree-Fock approximates the electronic structure of molecules by computing electron-repulsion integrals, a calculation that scales as $O(N^4)$ with the number of atoms. Parallelism is heavily limited by atomic operations, where multiple threads must update the same memory locations simultaneously. The results show significant inconsistency, with Mojo being 2.5x faster than CUDA at 256 atoms but 55x slower at 1024 atoms. On the MI300A, Mojo underperforms HIP in every case that completed, and the largest case (a=1024) produced no result at all. The authors do not define precisely what the missing value represents, presumably a timeout, noting only that they "observe abnormal behaviors in terms of performance for the 512 and 1024 cases" and that "Mojo largely underperforms the HIP implementation in all cases, which leads us to believe that we are still hitting a corner case in the language". The reported $\Phi = 0.92$ is misleading, as it averages out Mojo outperforming CUDA with Mojo underperforming HIP, obscuring the real picture [11] (Figure 14).

3) Overall

Mojo performs competitively with CUDA and HIP for memory-bound workloads, reaching near-parity on the MI300A and within a small margin on the H100.

Mojo Efficiency	NVIDIA H100	AMD MI300A
7-point stencil		
FP32	0.82	1.00
FP64	0.87	1.00
$\overline{\Phi}$ = 0.92		
BabelStream		
Copy	1.01	1.00
Mul	1.02	1.00
Add	1.01	1.00
Triad	1.01	1.00
Dot	0.78	1.00
$\overline{\Phi}$ = 0.96		
miniBUDE		
PPWI=8 wg=8	0.82	0.38
PPWI=4 wg=64	0.59	0.38
$\overline{\Phi}$ = 0.54		
Hartree–Fock		
a=1024 ngauss=6	17E-3	–
a=256 ngauss=3	2.52	7E-3
a=128 ngauss=3	2.52	8E-3
a=64 ngauss=3	2.33	8E-3
$\overline{\Phi}$ = 0.92		

Figure 10: Performance portability metric Φ for all four kernels on the NVIDIA H100 and AMD MI300A [11].

Gaps emerge on compute-bound kernels, where the lack of fast-math support and gaps in atomic operation support on AMD GPUs result in notable performance shortfalls [11].

F. Current Scope and Limitations

Mojo is still pre-1.0 software. The latest tagged release at the time of writing is v1.0.0b1, a beta whose language and APIs may still change before the eventual 1.0 release [15]. The examples in this paper target that version, and the limitations below should be read as a snapshot of a language still under active development rather than as permanent constraints.

Mojo currently targets single-node execution only, which limits its applicability to large-scale distributed HPC workloads. Several compiler features common in HPC also remain unavailable, most notably fast-math compilation, which relaxes floating-point precision rules to allow faster arithmetic and makes a significant performance difference in compute-bound workloads. More broadly, many HPC applications depend on highly optimised libraries such as BLAS, and portable Mojo equivalents are still being developed. Tooling support is similarly incomplete, with debugging limited to NVIDIA's Nsight toolchain and no officially supported profiling tool for AMD GPUs available at the time of the study [11].

V. Discussion

A. The Language Split and Developer Productivity

Mojo is often pitched as combining Python-like ease with C++ performance, but the reality of GPU programming in Mojo sits closer to the C++ end of that spectrum. Writing a kernel requires explicit thread indexing, compile-time problem sizes, and manual memory layouts, and Godoy et al. note that the learning curve and programming requirements remain fairly low-level [11]. Developers coming from Python who encounter Mojo's ownership model and compile-time constraints face a steeper transition than the language's surface syntax suggests [16]. More fundamentally, because Python interoperability is runtime-only, a developer working in Mojo still maintains a boundary between the Python world and the compiled Mojo world. The split is narrowed but reproduced in a different form, rather than eliminated.

B. Vendor Portability and Performance Cost

For memory-bound workloads the trade-off is favourable. As the benchmark results show, a single Mojo codebase delivers competitive performance on both the H100 and MI300A, which is a reasonable cost for eliminating vendor-specific rewrites. For compute-bound workloads the case is weaker, with miniBUDE reaching only 54% of vendor performance on average, driven largely by the absence of fast-math support [11]. Compared to Kokkos and SYCL, Mojo is less mature and currently lacks the high-level library coverage those approaches offer. The architectural difference is significant however. Mojo's portability is a compiler property rather than a library abstraction, meaning support for a new hardware target requires adding a lowering dialect rather than rewriting user code. This is an advantage that may compound as the ecosystem matures, in a way that library-based portability cannot.

Compilation time is a further trade-off. Mojo specializes kernels ahead of time, moving work that a just-in-time language like Julia defers to runtime into the build step instead, which brings it closer to the template-based C++ approaches that are themselves known for slow compilation. The MLIR foundation adds some overhead from its extensibility, but most of a Mojo program's compilation time is spent in the LLVM backend rather than in MLIR itself [17]. The proxy kernels in the benchmark study compile in only a few seconds, and a systematic comparison with Julia and C++ remains open [11].

C. Overall Assessment

Today, Julia handles the two-language split more effectively than Mojo, with a mature ecosystem and JIT compilation that reaches near-native performance without requiring a clean break between high-level and performance-critical code. Kokkos and SYCL handle vendor portability more effectively, with battle-tested library support and the BLAS integration that serious HPC workloads depend on. Mojo currently lags behind both on their respective strengths, with a steep low-level programming model, gaps in library coverage and compute-bound performance, and the practical limitations discussed above, including single-node execution, missing fast-math support, and incomplete tooling.

What Mojo offers that neither Julia nor Kokkos can is a single architectural foundation that structurally targets both problems at once. By being built directly on MLIR, hardware portability and progressive compilation are properties of the language itself rather than libraries added on top. If these gaps are addressed, the architecture is already in place to match or exceed what either approach offers individually. That combination has not existed before, and it remains Mojo's fundamental case for why it is worth watching. Having the right architecture and being production-ready are different things however, and whether Mojo's MLIR foundation will prove a lasting advantage over library-based approaches remains to be demonstrated at scale.

VI. Future Outlook

Several gaps identified in this paper are addressable in the near term, and the performance ones in particular are implementation issues rather than architectural ones. Fast-math support and the inconsistent atomic operation behavior on AMD are known missing features within MLIR's existing capabilities, not fundamental limitations of the approach. As Mojo matures, MLIR's AOT and JIT paths offer opportunities to close these gaps in ways library-based approaches cannot easily replicate. Broader adoption also depends on factors beyond performance. Mojo is currently source-available but not open source, and institutions building long-term HPC workflows are generally reluctant to commit to a closed language. Modular has stated that Mojo will become open-source by fall 2026 [11], which could significantly lower the barrier to adoption in academic and national lab settings. The language is also still in active development, and the specification, standard library, and GPU backend remain subject to breaking changes. Features critical for large-scale HPC such as multi-node support and portable high-level libraries are either absent or not yet mature.

The MLIR foundation gives Mojo an architectural advantage that alternatives do not have. Whether the ecosystem and governance can catch up to the technical promise is the open question. If they do, Mojo would be the first language to resolve both the two-language problem and vendor lock-in at the compiler level. The underlying infrastructure, MLIR, is already used in production ML systems and compilers across the industry, a solid foundation for Mojo to build on.

VII. References

[1] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," in *ACM SIGGRAPH 2008 classes*, in SIGGRAPH '08. New York, NY, USA: Association for Computing Machinery, Aug. 2008, pp. 1–14. doi: 10.1145/1401132.1401152.

[2] S. Atchley *et al.*, "Frontier: Exploring Exascale," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '23. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 1–16. doi: 10.1145/3581784.3607089.

[3] *ROCm/ROCm*. (Jun. 16, 2026). AMD ROCm™ Software. Accessed: Jun. 16, 2026. [Online]. Available: https://github.com/ROCm/ROCm

[4] *ROCm/hip*. (Jun. 16, 2026). AMD ROCm™ Software. Accessed: Jun. 16, 2026. [Online]. Available: https://github.com/ROCm/hip

[5] J. Bezanson, A. Edelman, S. Karpinski, and V. Shah, "Julia: A Fresh Approach to Numerical Computing." Accessed: Apr. 28, 2026. [Online]. Available: https://epubs.siam.org/doi/epdf/10.1137/141000671

[6] J. Bezanson, S. Karpinski, V. Shah, and A. Edelman, "Why We Created Julia." Accessed: Apr. 28, 2026. [Online]. Available: https://julialang.org/blog/2012/02/why-we-created-julia/

[7] T. Besard, C. Foket, and B. D. Sutter, "Effective Extensible Programming: Unleashing Julia on GPUs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 4, pp. 827–841, Apr. 2019, doi: 10.1109/TPDS.2018.2872064.

[8] C. R. Trott *et al.*, "Kokkos 3: Programming Model Extensions for the Exascale Era," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 805–817, Apr. 2022, doi: 10.1109/TPDS.2021.3097283.

[9] R. Keryell, R. Reyes, and L. Howes, "Khronos SYCL for OpenCL: A tutorial," in *Proceedings of the 3rd International Workshop on OpenCL*, in IWOCL '15. New York, NY, USA: Association for Computing Machinery, May 2015, p. 1. doi: 10.1145/2791321.2791345.

[10] C. Lattner *et al.*, "MLIR: A Compiler Infrastructure for the End of Moore's Law." Accessed: Apr. 28, 2026. [Online]. Available: http://arxiv.org/abs/2002.11054

[11] W. F. Godoy *et al.*, "Mojo: MLIR-Based Performance-Portable HPC Science Kernels on GPUs for the Python Ecosystem," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2025, pp. 2114–2128. doi: 10.1145/3731599.3767573.

[12] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization, 2004. CGO 2004.*, IEEE, 2004, pp. 75–86. Accessed: Apr. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1281665/

[13] J. E. Stone, D. Gohara, and G. Shi, "OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, vol. 12, no. 3, pp. 66–73, May 2010, doi: 10.1109/MCSE.2010.69.

[14] "Mojo Manual | Mojo." Accessed: May 08, 2026. [Online]. Available: https://mojolang.org/docs/manual/

[15] "Mojo releases | Mojo." Accessed: Jun. 16, 2026. [Online]. Available: https://mojolang.org/releases/

[16] P. D. Akre and U. Pacharaney, "A Comprehensive Review of Mojo: A High-Performance Programming Language," in *2025 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, Jan. 2025, pp. 861–867. doi: 10.1109/ICMCSI64620.2025.10883176.

[17] M. Amini and J. Niu, "How slow is MLIR," presented at the EuroLLVM, Jun. 21, 2024.

VIII. Appendix

The complete Mojo 1.0.0b1 [15] matrix multiplication implementation used throughout this paper, including host setup, device memory allocation, and kernel dispatch, as well as the full MLIR intermediate representations at each lowering stage (linalg, affine, scf, and LLVM IR), are available in the accompanying repository at https://github.com/Jan-Kluger/mojo_mlir_seminar_mpcdf.

A. Benchmark Charts

The following figures show the per-kernel benchmark results summarised by the performance portability metric Φ in Figure 10.

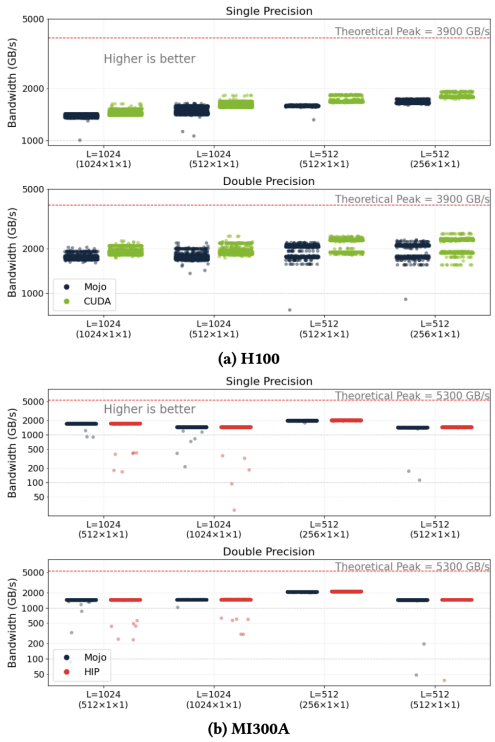

Figure 11: Mojo vs CUDA and HIP performance for the seven-point stencil kernel on the NVIDIA H100 and AMD MI300A [11].

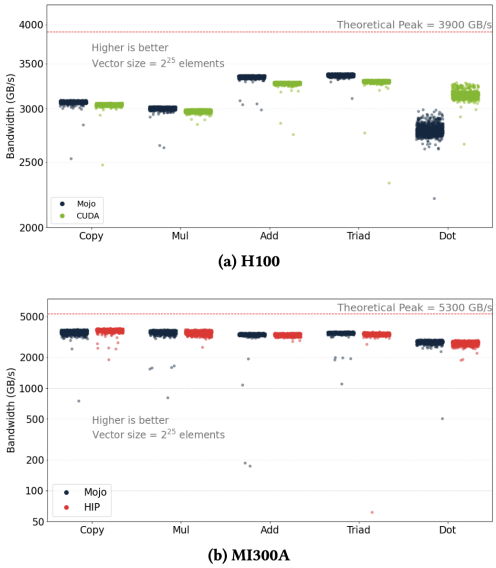

Figure 12: Mojo vs CUDA and HIP performance for BabelStream kernels on the NVIDIA H100 and AMD MI300A [11].

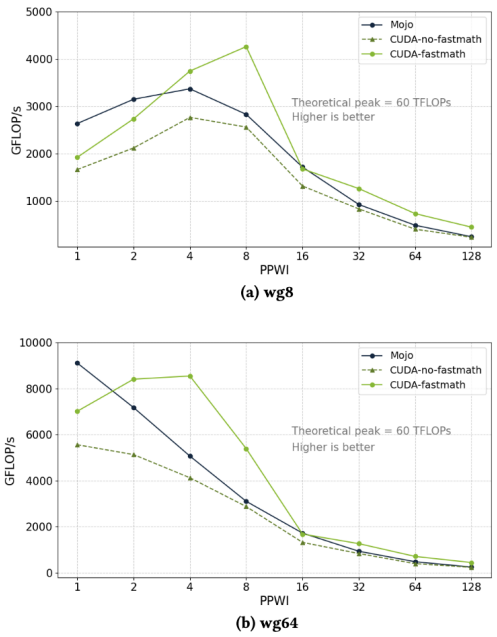

Figure 13: Mojo vs CUDA and HIP performance for miniBUDE on the NVIDIA H100 and AMD MI300A [11].

Kernel execution duration (ms)	NVIDIA H100 Mojo	NVIDIA H100 CUDA	AMD MI300A Mojo	AMD MI300A HIP
a=1024 ngauss=6	147,250	2,652	–	846
a=256 ngauss=3	187	472	25,266	178
a=128 ngauss=3	21	53	2,765	23
a=64 ngauss=3	3	7	436	4

Figure 14: Mojo vs CUDA and HIP kernel execution times for Hartree-Fock on the NVIDIA H100 and AMD MI300A [11].